Isaac Sim ↔ ROS2 Jazzy Setup Summary (UR5e)

2026-07-08

Goal

- Connect a UR5e arm simulated in Isaac Sim 6.0.1 (running on Windows) to ROS2 Jazzy (running in WSL2), and drive joint movements from ROS2.

Environment

- Isaac Sim 6.0.1 runs natively on Windows, not inside WSL2.
- ROS2 Jazzy runs natively inside WSL2 (Ubuntu 24.04).
- WSL2 networking mode is "mirrored" — WSL2 shares Windows' IP addresses and loopback interface.

The core problem

- ROS2 topic discovery relies on UDP multicast by default.
- This is a known, NVIDIA-confirmed bug: UDP multicast does not reliably cross the Windows ↔ WSL2 boundary under mirrored networking mode, even though ordinary unicast traffic (ping, TCP, plain UDP) works fine.
- Result: `ros2 topic list` in WSL2 only ever showed default topics — nothing from Isaac Sim — regardless of how correct the Isaac Sim graph setup was.
- NVIDIA's officially documented fix is switching WSL2 to NAT mode plus manual port forwarding (`netsh portproxy`). We used a less disruptive community workaround instead.

The fix

- Created a custom Fast-DDS configuration profile that disables multicast entirely and forces all DDS traffic — both discovery and actual message data — over loopback (127.0.0.1), which mirrored-mode WSL2 shares reliably with Windows.
- Applied identically on both sides via an environment variable (`FASTRTPS_DEFAULT_PROFILES_FILE`) pointing to matching profile files on WSL2 and Windows.
- Two mistakes found and corrected along the way:
    - Initially only routed discovery traffic through loopback, not actual message data — this caused one-directional communication (Isaac Sim → ROS2 worked, ROS2 → Isaac Sim did not) until both were forced through loopback.

    - A workaround XML found online contained invalid configuration tags for our Fast-DDS version, which silently failed to parse and fell back to defaults — had to simplify it to just the valid options.
- Also matched the DDS middleware implementation (`RMW_IMPLEMENTATION`) on both sides to `rmw_fastrtps_cpp`, since WSL2 defaulted to a different one than Isaac Sim.
- A Fast-DDS Discovery Server approach was tried first as an alternative fix; it didn't resolve the issue for us and was abandoned in favor of the loopback profile.

Isaac Sim side: Action Graph

- Built an Action Graph reading the UR5e's live joint state and publishing it to ROS2, and separately subscribing to ROS2 joint commands and applying them to the arm via an Articulation Controller.
- Gotcha: the articulation root (the prim that identifies the robot to the physics engine) was not where expected — it was on a child prim named `root_joint`, not the robot's top-level object. Found by asking the USD stage directly which prim had the articulation API applied, rather than hunting through the scene tree.
- Gotcha: even after the graph was wired correctly and data was confirmed flowing all the way to the Articulation Controller, the arm still didn't move. Isaac Sim's console showed a "simulation view invalidated" error — caused by editing the graph/stage while the simulation was playing, which corrupts an internal physics engine handle. Fixed with a clean Stop → Play cycle.

ROS2 package delivered

- Created a ROS2 package (`ur5e_isaac_control`) with a joint command publisher node supporting three modes:
    - Move to an explicit target pose (smoothly interpolated, not a sudden jump)
    - A quick two-pose demo sweep for sanity-checking the connection
    - A preset pick-and-place mimic sequence (approach, reach down, lift, transfer, reach down, lift, return home)
- The node waits for both a live joint-state reading and a confirmed subscriber match before sending commands, since discovery over the loopback profile isn't instantaneous and early messages can otherwise be silently dropped.
- Confirmed the UR5e's joints have real position-control gains, so Isaac Sim holds the last commanded position on its own — no need to publish continuously to keep the arm in place.

Outcome

- Verified end-to-end: live joint state streaming from Isaac Sim into ROS2, and commanded joint positions from ROS2 actually moving the arm in the Isaac Sim viewport, including the full pick-and-

place mimic sequence.

Incidental fixes made along the way

- The ROS2 workspace was renamed from `matlabros_ws` to `isaac_sim_ros`; a stale reference to the old path in the shell startup config was found and corrected.
- WSL2's default ROS2 middleware implementation didn't match Isaac Sim's; corrected for consistency and to rule out interoperability issues.

> Full technical detail, including exact configuration file contents and file paths, is kept in `SETUP_NOTES.md` in the isaac_sim_ros repository.